GameCore - Trabalho 3

Documentação do processo e das decisões | Persistência local com Room

Integrantes: Gabriel Henrique Alves Raatz e Matheus Pupia

Repositório: github.com/GabrielRaatz18/GameCorePart3

1. Do Trabalho 2 para o Trabalho 3

No Trabalho 2, as avaliações e a Lista de desejos eram listas mantidas em memória. Era possível adicionar e remover itens durante o uso, mas eles desapareciam quando o aplicativo era fechado. Nesta etapa, mantivemos as telas e passamos a salvar esses dois tipos de dado em um banco SQLite local usando Room. Ao abrir as telas, os registros são lidos novamente do banco.

O carrinho e a Biblioteca continuam temporários. A compra simulada coloca o jogo na Biblioteca durante a sessão para liberar o formulário de avaliação. O catálogo de jogos permanece no código; ele não foi transformado em tabela nesta etapa.

2. Estrutura do banco

Entidade / tabela	Campos	Decisão
ReviewData **avaliacoes**	id, gameId, author, rating, text, helpful	O id identifica a avaliação; gameId relaciona o registro ao jogo. Os demais campos guardam autor, nota, comentário e votos úteis.
WishlistEntry **lista_desejos**	id, gameId, adicionadoEm	Guardamos só o identificador do jogo e a data da inclusão. Nome, capa e preço continuam no catálogo, sem duplicação no banco.

As duas entidades usam **@Entity** e chave primária **id: Int = 0** com **@PrimaryKey(autoGenerate = true)**. A Lista de desejos tem índice único em gameId, impedindo a inclusão duplicada do mesmo jogo.

A classe **AppDatabase** reúne as entidades e expõe os DAOs. A instância é criada por Room.databaseBuilder e reutilizada com companion object, @Volatile e synchronized. O banco está na versão 2; a migração 1 para 2 adicionou a tabela de usuários sem apagar avaliações e desejos.

> O projeto também possui cadastro e login locais, pedidos como funcionalidade adicional. Eles usam a entidade Usuario. O foco desta etapa, conforme o enunciado, é a persistência de avaliações e da Lista de desejos; não há servidor nem sincronização na nuvem.

3. DAOs e consultas

Criamos **ReviewDao** e **WishlistDao**, um para cada entidade principal. Ambos possuem métodos suspend de inserir (@Insert), buscar todos (@Query), atualizar (@Update) e remover (@Delete). Também há busca por id. Em ReviewDao, buscarPorJogoId(gameId) seleciona apenas as avaliações do jogo aberto; em WishlistDao, a mesma consulta localiza o item a remover.

As operações são iniciadas pelo GameCoreViewModel em viewModelScope.launch(Dispatchers.IO), evitando acesso ao SQLite na thread da interface. Após a resposta do banco, o estado observado pelo Compose é atualizado. As operações têm tratamento de erro com try/catch e Log.e.

4. CRUD nas telas e detalhe

• **Buscar:** Lista de desejos e Avaliações carregam os registros ao abrir, usando LaunchedEffect. A lista de avaliações é filtrada pelo jogo.

• **Inserir:** o botão do detalhe do jogo salva o item desejado; o formulário de avaliação publica uma nova linha no Room.

• **Atualizar:** no detalhe de uma avaliação, nota e texto podem ser alterados e salvos pelo DAO.

• **Remover:** os botões de remoção apagam a linha correspondente do banco. WishlistDao contém atualizar, mas a Lista de desejos não tem formulário de edição, pois guarda apenas o jogo e a data de inclusão.

A tela de detalhe da avaliação recebe somente **reviewId** pela navegação. O ViewModel chama ReviewDao.buscarPorId(id) para ler o registro diretamente do banco, sem passar o objeto inteiro entre telas.

5. Dificuldades e soluções

Gabriel e Matheus tiveram dificuldade para decidir como organizar a estrutura do banco. A solução foi separar avaliações e Lista de desejos em duas tabelas e manter o catálogo de jogos no código. Assim, a Lista de desejos salva o gameId e usa esse identificador para exibir os dados do jogo.

Na migração das listas em memória para o Room, a Lista de desejos chegou a mostrar um jogo antes de o banco confirmar a gravação. Ajustamos o fluxo para atualizar a tela somente depois de inserir e buscar os registros; inclusões pendentes também evitam toques repetidos. Na remoção, a lista é relida depois do delete.

A primeira configuração do Room também apresentou erro de compilação com a combinação de Kotlin e KSP do projeto. A atualização da dependência para Room 2.8.5 resolveu a geração do código.

6. Evidências e apresentação

Vídeo da equipe: assistir à gravação do GameCore no YouTube. A gravação mostra cadastro, navegação, Lista de desejos e uma avaliação salva.

> O teste de persistência aparece no trecho final do vídeo: aproximadamente em 1:39, a avaliação está na tela; o aplicativo sai para a tela inicial do celular e é aberto novamente; aproximadamente em 1:59, a mesma avaliação reaparece. Isso mostra o registro após a reabertura do app.

Na apresentação em sala, podemos repetir o fluxo de publicar uma avaliação, alterar nota ou texto, remover um registro e reabrir o aplicativo para mostrar o dado que continuou salvo. Não devemos limpar os dados nem desinstalar o app durante o teste, pois isso apagaria o banco local.

7. Como executar

Abra o projeto no Android Studio, sincronize o Gradle e execute o módulo app em aparelho ou emulador com Android 7.0 (API 24) ou superior. No primeiro acesso, cadastre uma conta. Para testar avaliações, conclua uma compra simulada e abra o jogo pela Biblioteca; para a Lista de desejos, use o botão no detalhe do jogo.